

Lembrando nosso dataset

Voltamos ao mesmo `notas_estudo.csv` da Aula 04 — 80 alunos fictícios, horas de estudo por semana e nota da prova.

Mesma base, problema diferente — vamos prever aprovação, não a nota exata.



Vamos ver na prática

```
import pandas as pd

df = pd.read_csv("notas_estudo.csv")
df["aprovado"] = (df["nota_prova"] >= 6).astype(int)

df[["horas_estudo_semana", "nota_prova", "aprovado"]].head()
```



**Como prever se um aluno vai passar ou não,
em vez de prever a nota exata?**



Por que não usar a reta de novo

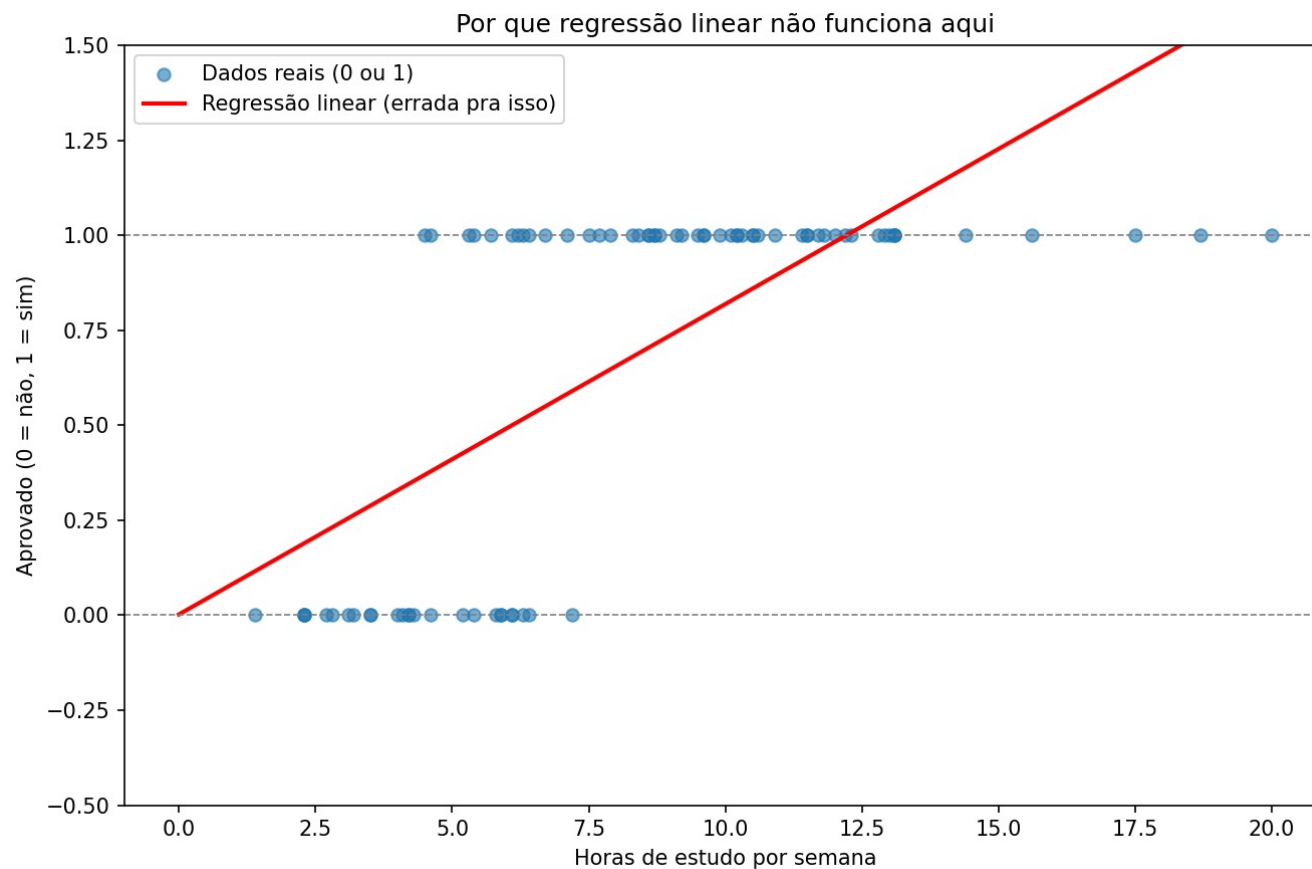
Probabilidade só faz sentido entre 0 e 1 — mas uma reta não tem limite nenhum, pode prever qualquer número.

Se tentássemos usar a regressão linear pra prever 'aprovado' (0 ou 1), a reta ultrapassaria esses limites facilmente.

Precisamos de uma função que sempre devolva algo entre 0 e 1.



O problema, visualmente



Vamos ver na prática

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

X = df[["horas_estudo_semana"]]
y = df["aprovado"]

modelo_linear = LinearRegression().fit(X, y)
x_linha = np.linspace(0, 20, 100).reshape(-1, 1)
y_reta = modelo_linear.predict(x_linha)

plt.scatter(df["horas_estudo_semana"], df["aprovado"], alpha=0.6)
plt.plot(x_linha, y_reta, color="red", label="Regressao linear")
plt.legend()
plt.show()
```



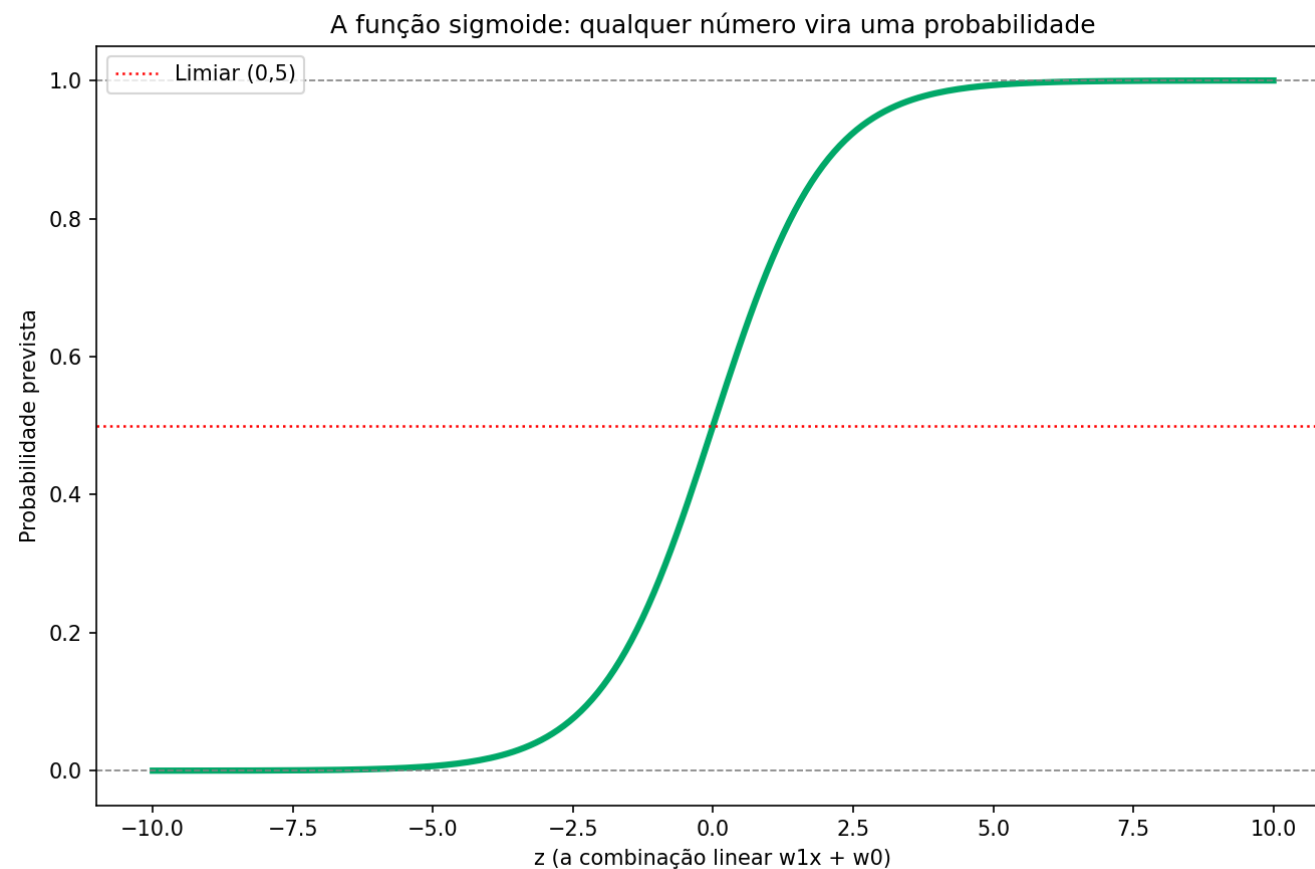
E se existisse uma função que resolvesse isso?

Existe: a função sigmoide. Ela pega qualquer número (positivo, negativo, gigante, minúsculo) e sempre devolve um valor entre 0 e 1.

Perfeito pra representar uma probabilidade.



A curva sigmoide



Vamos ver na prática

```
z = np.linspace(-10, 10, 200)
sigmoide_z = 1 / (1 + np.exp(-z))

plt.plot(z, sigmoide_z, color="green", linewidth=3)
plt.axhline(0.5, color="red", linestyle=":")
plt.xlabel("z")
plt.ylabel("Probabilidade")
plt.show()
```



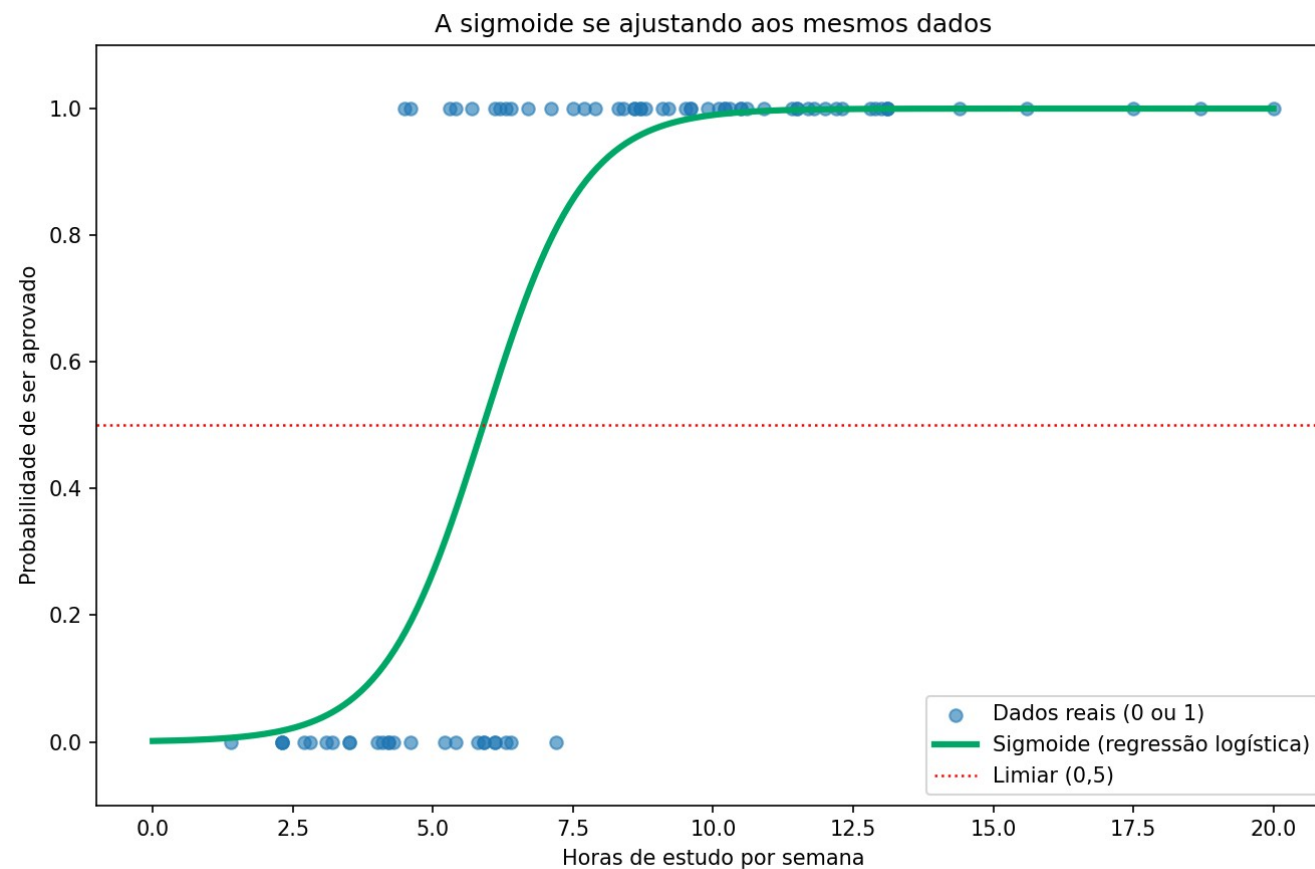
A fórmula da sigmoide

$\sigma(z) = 1 / (1 + e^{-z})$, onde $z = w_1x + w_0$ — a mesma combinação linear que já conhecemos da Aula 04.
z pode ser qualquer número; a sigmoide ‘espreme’ esse número pra caber entre 0 e 1.

Regressão logística é regressão linear, com uma etapa extra no final.



A sigmoide ajustada aos dados reais



Vamos ver na prática

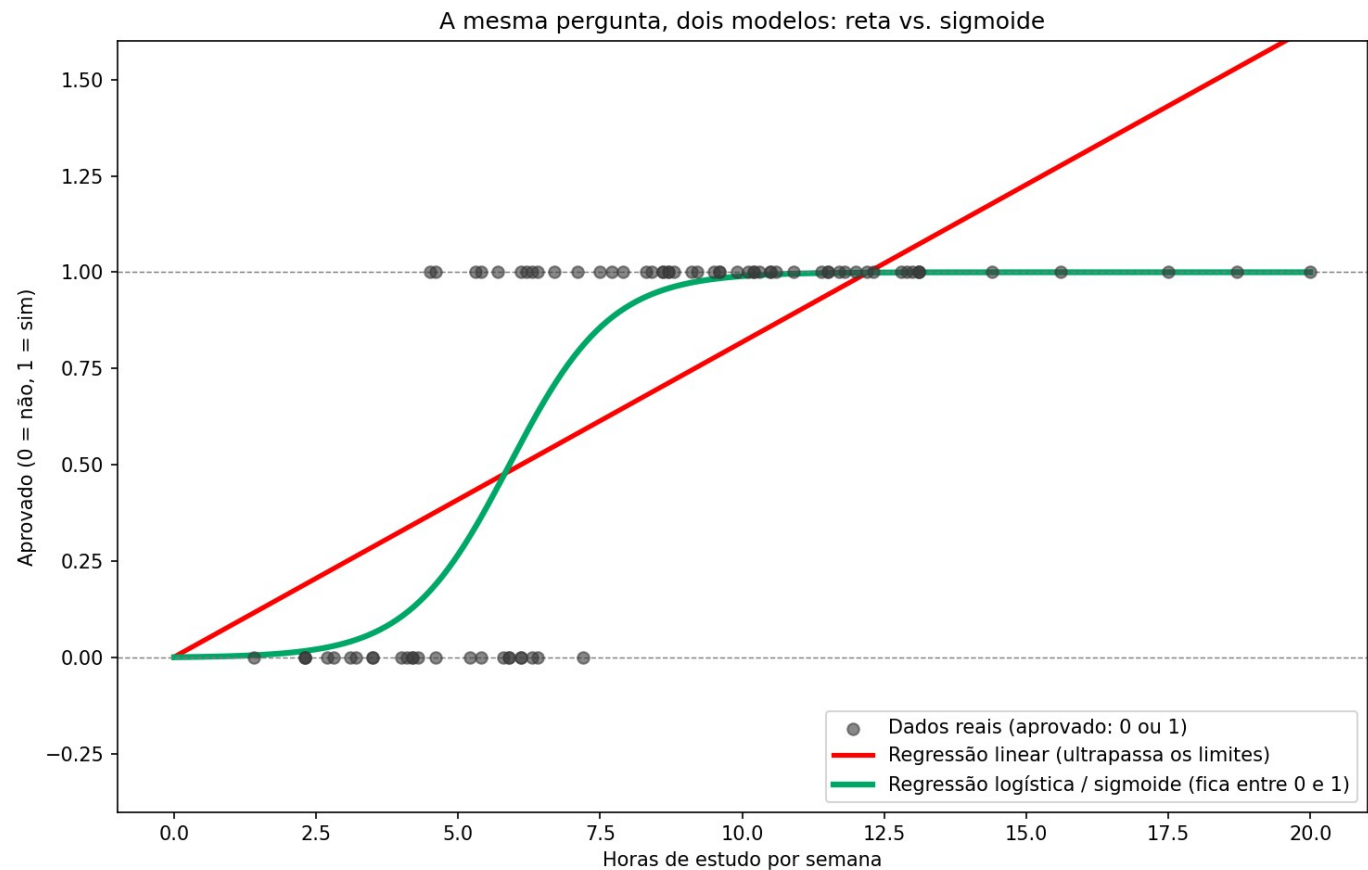
```
from sklearn.linear_model import LogisticRegression

modelo_log = LogisticRegression().fit(X, y)
prob_linha = modelo_log.predict_proba(x_linha)[: , 1]

plt.scatter(df["horas_estudo_semana"], df["aprovado"], alpha=0.6)
plt.plot(x_linha, prob_linha, color="green", linewidth=3, label="Sigmoide")
plt.axhline(0.5, color="red", linestyle=":")
plt.legend()
plt.show()
```



Reta vs. sigmoide, lado a lado



Vamos ver na prática

```
plt.scatter(df["horas_estudo_semana"], df["aprovado"], alpha=0.6, color="gray")  
plt.plot(x_linha, y_reta, color="red", label="Regressao linear")  
plt.plot(x_linha, prob_linha, color="green", linewidth=3, label="Regressao logistica")  
plt.legend()  
plt.show()
```



Como o algoritmo encontra os melhores pesos

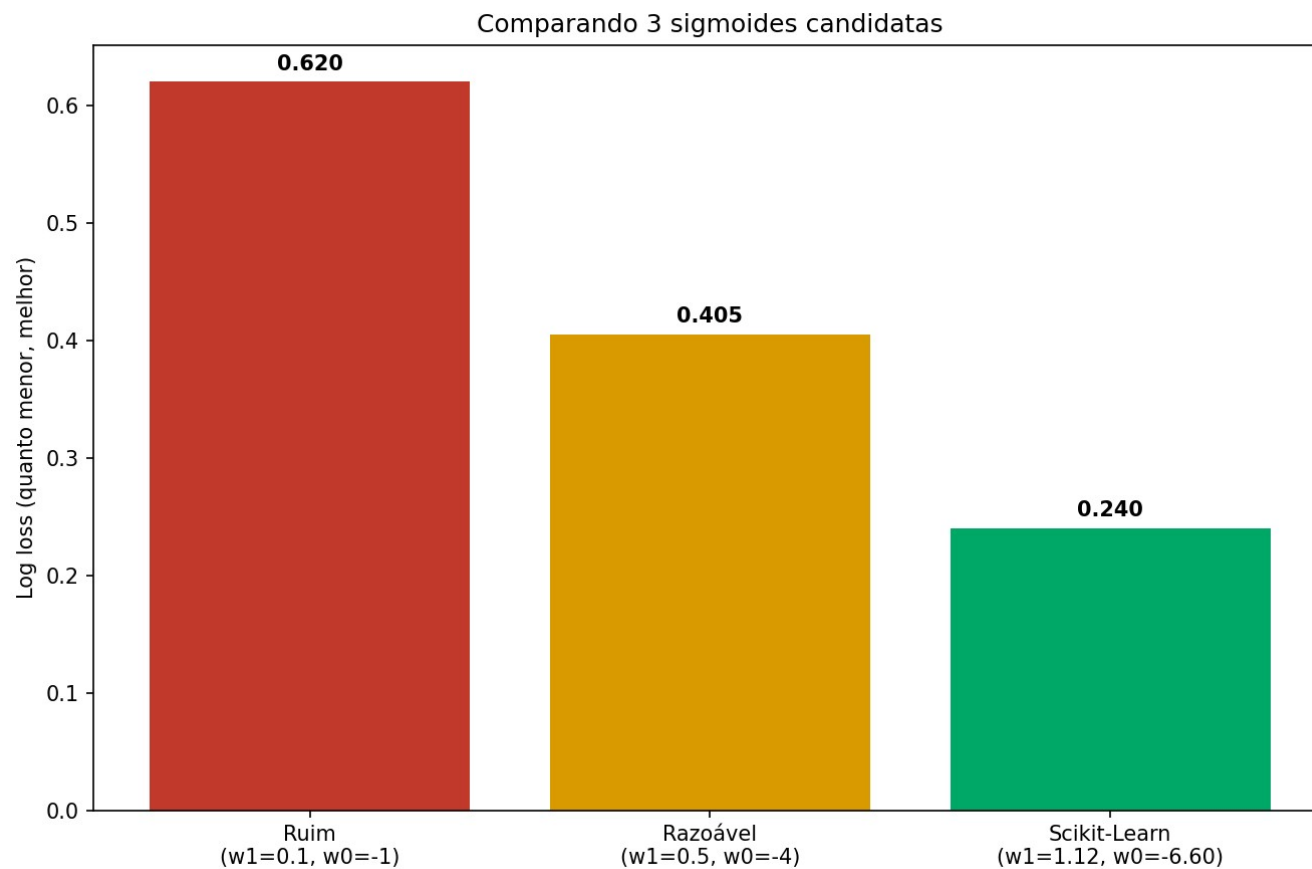
Assim como na regressão linear, existe um 'erro' sendo minimizado — aqui ele se chama log loss, e penaliza mais forte os erros feitos com confiança.

Diferente da regressão linear (fórmula exata), aqui o algoritmo já faz uma busca iterativa — a mesma ideia de tentar, errar e ajustar que vamos ver com mais força futuramente.

Essa busca iterativa vai reaparecer, com mais detalhes, futuramente no curso.



Comparando 3 sigmoides candidatas



Vamos ver na prática

```
def log_loss(w1, w0, x, y):  
    z = w1 * x + w0  
    p = 1 / (1 + np.exp(-z))  
    p = np.clip(p, 1e-10, 1 - 1e-10)  
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))  
  
x_arr = df["horas_estudo_semana"].values  
y_arr = df["aprovado"].values  
  
candidatas = {  
    "Ruim": (0.1, -1),  
    "Razoavel": (0.5, -4),  
    "Scikit-Learn": (modelo_log.coef_[0][0], modelo_log.intercept_[0]),  
}  
valores = [log_loss(w1, w0, x_arr, y_arr) for w1, w0 in candidatas.values()]  
  
plt.bar(list(candidatas.keys()), valores, color=["red", "orange", "green"])  
plt.show()
```



O limiar de decisão

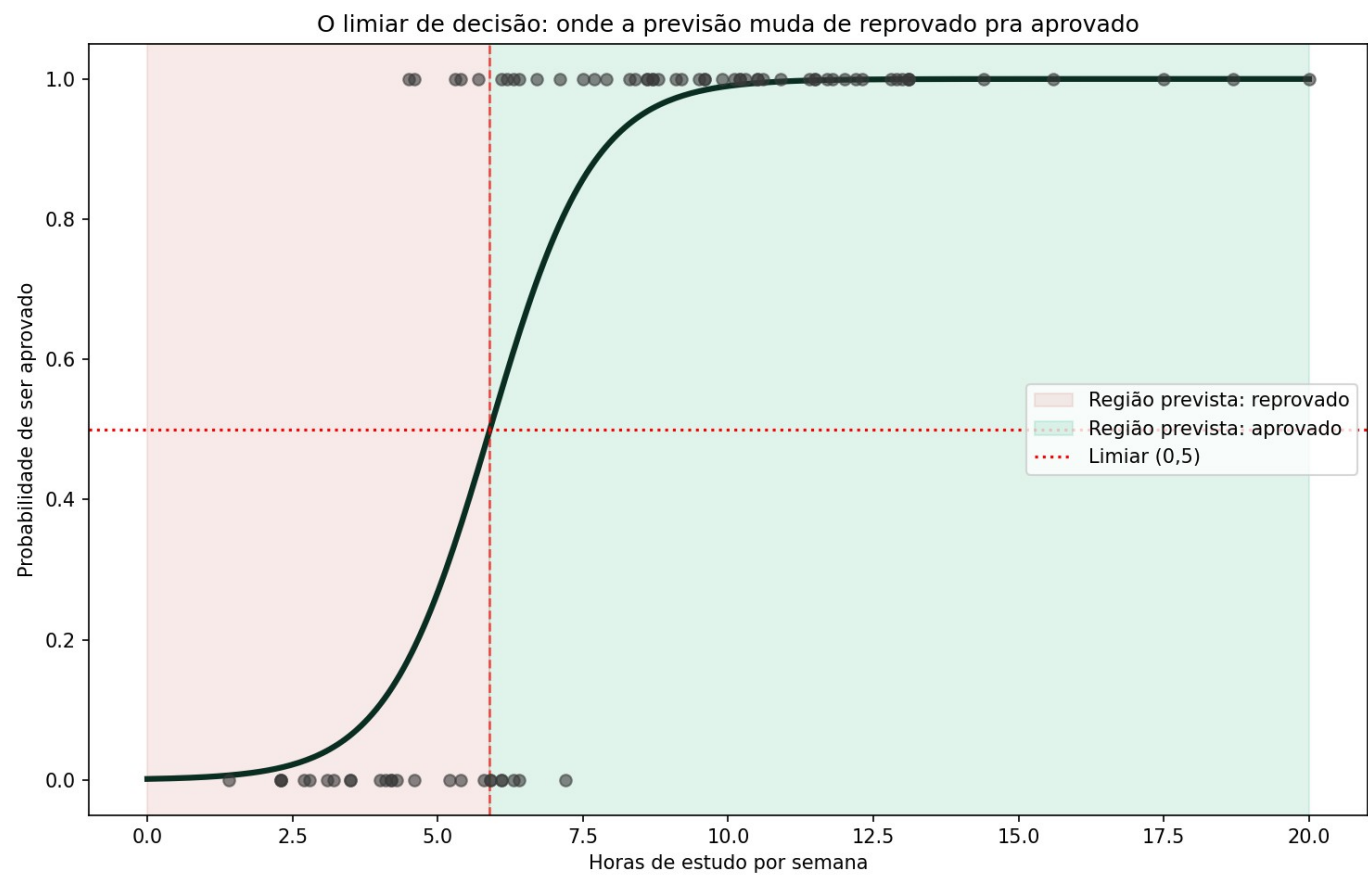
A sigmoide devolve uma probabilidade (tipo 0,73) — precisamos de uma regra pra virar decisão.

O padrão: se a probabilidade for $\geq 0,5$, prevemos 'aprovado'; senão, 'reprovado'.

Esse limiar pode ser ajustado — vamos ver isso com mais detalhe na Sessão 2, com precisão e recall.



Onde a decisão muda



Vamos ver na prática

```
idx_frenteira = np.argmin(np.abs(prob_linha - 0.5))
x_frenteira = x_linha.flatten()[idx_frenteira]
print(f"Frenteira: {x_frenteira:.2f} horas")

plt.axvspan(0, x_frenteira, color="red", alpha=0.1)
plt.axvspan(x_frenteira, 20, color="green", alpha=0.12)
plt.scatter(df["horas_estudo_semana"], df["aprovado"], alpha=0.6, color="gray")
plt.plot(x_linha, prob_linha, color="black", linewidth=3)
plt.show()
```



O fluxo no Scikit-Learn

Mesma gramática de sempre: `fit()` treina, `predict()` prevê a classe final.

A novidade: `predict_proba()` devolve a probabilidade, não só a classe — útil pra entender o quão confiante o modelo está.

`predict()` e `predict_proba()` respondem perguntas diferentes: 'qual classe?' vs 'quão certo você está?'



Vamos ver na prática

```
exemplo = pd.DataFrame({"horas_estudo_semana": [3, 6, 12]})
```

```
print("predict():", modelo_log.predict(exemplo))
```

```
print("predict_proba():")
```

```
print(modelo_log.predict_proba(exemplo))
```

```
aluno = pd.DataFrame({"horas_estudo_semana": [5]})
```

```
probabilidades = modelo.predict_proba(aluno)
```

```
print(probabilidades)
```



Treino e teste, com um cuidado a mais

Testamos vários seeds sem cuidado especial, e a proporção de aprovados no teste variou de 50% a 81% — bem longe dos 67,5% reais.

stratify=y resolve isso: garante que a proporção de cada classe fique igual no treino e no teste, não importa o seed.

Isso não existia em regressão — só faz sentido quando o y é uma categoria.



Vamos ver na prática

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7, stratify=y
)

print("Treino:", y_train.mean().round(3))
print("Teste:", y_test.mean().round(3))
```



Vamos ver na prática

```
modelo_final = LogisticRegression()  
modelo_final.fit(X_train, y_train)  
  
print("w1:", modelo_final.coef_[0][0])  
print("w0:", modelo_final.intercept_[0])
```



Um primeiro olhar pra acurácia

`accuracy_score` mede a proporção de previsões certas — o jeito mais simples de avaliar um classificador.

Vamos aprofundar bastante em métricas (precisão, recall, F1) na Sessão 2 — por hoje, só uma primeira noção.

Acurácia sozinha pode enganar quando as classes são desbalanceadas — mais um gancho pra Sessão 2.



Vamos ver na prática

```
from sklearn.metrics import accuracy_score

pred = modelo_final.predict(X_test)
acuracia = accuracy_score(y_test, pred)
print(f"Acuracia: {acuracia:.2%}")
```



Da reta à sigmoide, da probabilidade à decisão

Vimos por que a reta não serve pra classificação, como a sigmoide resolve isso, como o algoritmo busca os melhores pesos, o limiar de decisão, e o cuidado extra com stratify no treino/teste.

Hoje vimos a versão de 2 classes — mas o mesmo algoritmo se estende pra mais categorias.